AWS-Compliant PPTX Generation Skill: Complete Development Framework

1 Executive Summary

本报告提供了一套完整的、可直接投入生产的 AWS 品牌合规 PPTX 自动生成 Skill 开发框架，涵盖从品牌标准解析到代码实现、部署验证的全链路方案。

该 Skill 采用**三层架构**（输入层 → 引擎层 → 输出层）[1]，以 python-pptx 1.0.2 为渲染核心 [2]，FastAPI 提供 REST 服务接口 [3]，Pydantic 实现类型安全的 JSON Schema 校验 [4]。核心设计使用 **Strategy 模式**将每种幻灯片类型（封面、内容、架构图、表格等 12 种标准类型）封装为独立 Builder [5]，通过 Orchestrator 统一调度，新增幻灯片类型无需修改现有代码。模板采用真实 .pptx 文件作为起点，保留 Slide Master 和 Layout 的视觉配置，实现设计与逻辑分离——视觉变更无需代码部署 [6, 1]。

品牌合规方面，所有输出严格遵循 AWS 官方标准：16:9 宽屏（13.33 × 7.5 英寸）[7, 8]，Squid Ink (#232F3E) 深色背景 [9]，Smile Orange (#ED7100) 强调色 [9]，Amazon Ember 字体族 [10]，架构图标签统一使用 12pt Arial [11]。报告提供了完整的八色服务分类色板、排版层级规范及架构图绘制规则（2pt 线宽、Open Arrow Size 4 连接器、0.05” 嵌套缓冲）[11]。

最关键的可操作洞察是：该 Skill 通过 **Manifest 驱动的四阶段验证管线**（初始化 → 提取清单 → 边界内生成 → 输出校验）[12] 确保每份输出的品牌一致性，配合 Placeholder idx 稳定性保证 [13] 和 BytesIO 流式 I/O [6]，可无缝集成 S3/ECS/API Gateway 的 AWS 原生部署架构，实现企业级可扩展的自动化演示文稿生成服务。

2 Introduction and Skill Overview

This document provides a complete development framework for building an **AWS-compliant PPTX generation skill** — a programmatic service that produces professional PowerPoint presentations adhering to official AWS brand standards. The skill leverages python-pptx 1.0.2 (MIT-licensed, Production/ Stable) [2] as its rendering engine, combined with a three-layer architecture separating template management, data binding, and output generation [1].

Technology Stack: The core stack consists of python-pptx for PPTX rendering [2], FastAPI for the service API layer [3], Pydantic for input validation and schema enforcement [4], and Docker for containerized deployment. The system supports Python ⩾3.8 and operates without requiring Microsoft Office installation [2].

Scope and Capabilities: The skill generates presentations in 16:9 widescreen format (13.33 × 7.5 inches) [14, 7, 8] using the AWS dark theme centered on Squid Ink (#232F3E) backgrounds [9, 15]. It supports standard AWS slide types including Title/Cover, Architecture Diagram, and Q&A/Closing slides [16], with full compliance to AWS typography (Amazon Ember as the master brand font [10]), official service category colors [9], and architecture icon guidelines [16].

Design Philosophy: Following the manifest-based template compiler pattern [12], the skill initializes from a real .pptx template file containing pre-configured slide masters and layouts [17, 6], then generates slides within approved boundaries using a strategy pattern that dispatches to type-specific builders [5]. This separation of design and logic ensures that visual changes don’ t require code deployments [1], while Pydantic schemas guarantee type-safe configuration input [4].

Chart Explanation: The architecture follows the dominant three-layer pattern [1] with an Input Layer (JSON config validated by Pydantic [4], data sources, and template .pptx), an Engine Layer using python-pptx with the strategy pattern for slide type dispatch [5], and an Output Layer producing AWS-compliant presentations with post-generation validation [3, 12].

3 AWS Brand Design Standards

AWS maintains a rigorous visual identity system documented in the official Architecture Icons Deck (Release 22-2025.07.31) and brand guidelines. All presentations must comply with these standards to ensure consistency across AWS communications.

3.1 Color System

The AWS color palette is anchored by two primary colors: **Squid Ink (#232F3E)** as the dark brand background and **Smile Orange (#ED7100)** as the primary accent [9]. Eight service category colors are defined for architecture diagrams and visual differentiation [9]:

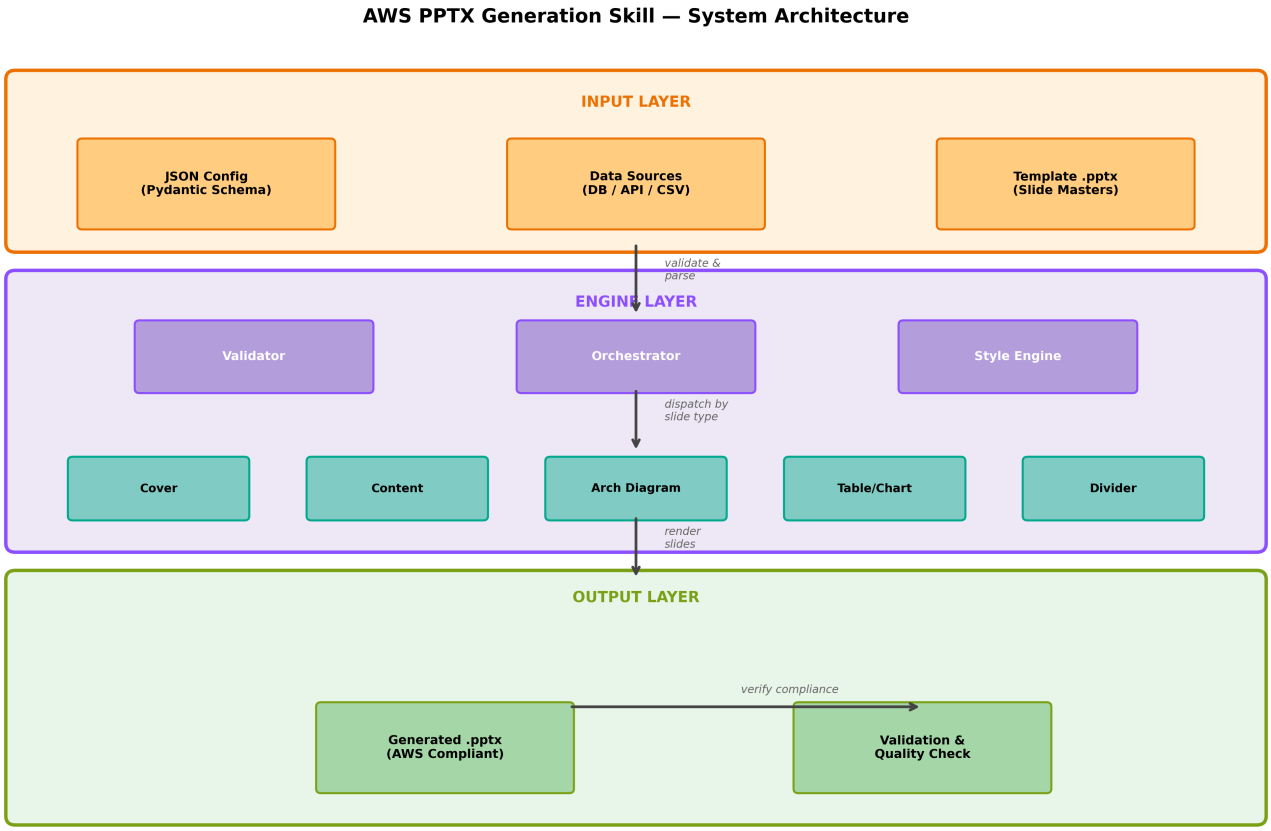

Figure 1: AWS PPTX Skill Architecture

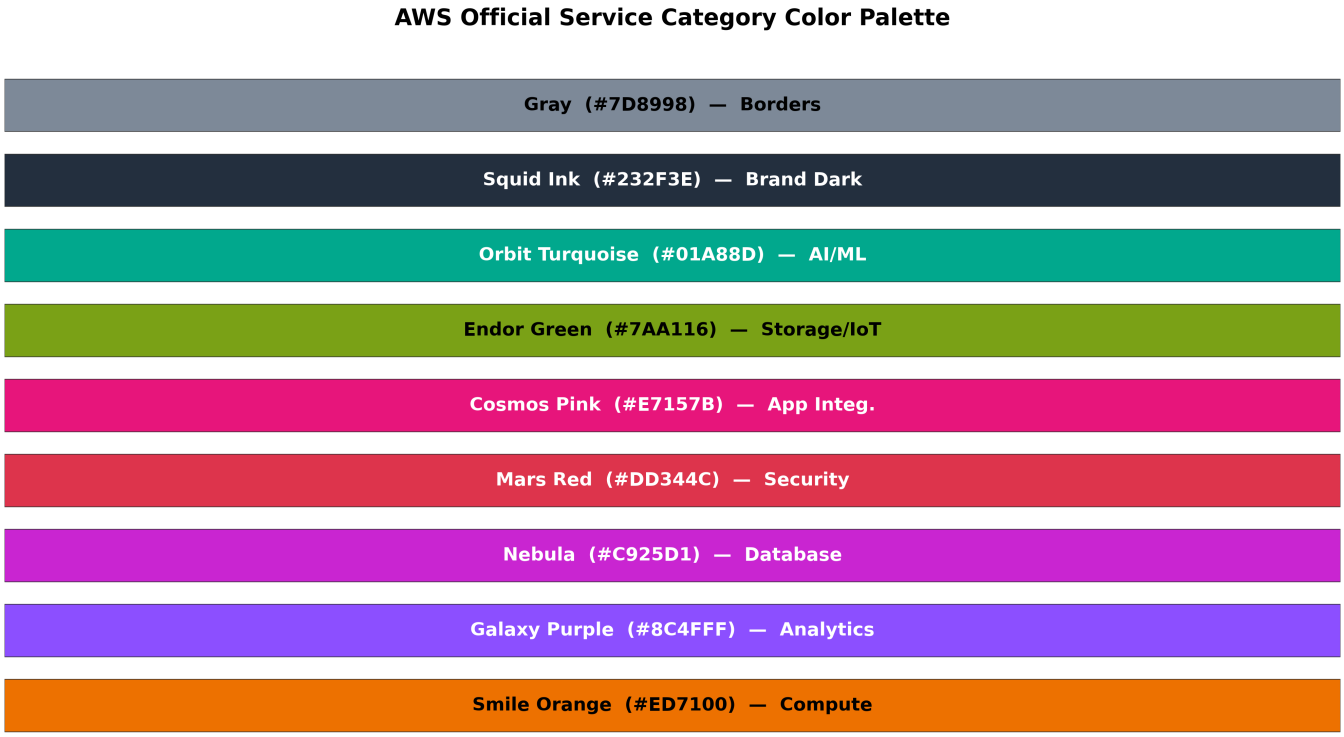

Figure 2: AWS Color Palette

Chart Explanation: The chart displays all official AWS service category colors from the awslabs icon repository [9]. Smile Orange (#ED7100) maps to Compute/Containers [9], Galaxy Purple (#8C4FFF) to Analytics/Networking, Nebula (#C925D1) to Database/Developer Tools, Mars Red (#DD344C) to Security, Cosmos Pink (#E7157B) to App Integration, Endor Green (#7AA116) to Storage/IoT, and Orbit Turquoise (#01A88D) to AI/Migration [9]. Squid Ink (#232F3E) serves as the primary dark background [9].

Color Name	Hex Code	Service Categories
Smile Orange	#ED7100	Compute, Containers, Blockchain, Media Services [9]
Galaxy Purple	#8C4FFF	Analytics, Games, Networking, Serverless [9]
Nebula	#C925D1	Database, Developer Tools, Satellite [9]
Mars Red	#DD344C	Security, Business Apps, Front-End Web [9]
Cosmos Pink	#E7157B	App Integration, Management & Governance [9]
Endor Green	#7AA116	Storage, IoT, Cloud Financial Management [9]
Orbit Turquoise	#01A88D	AI, End User Computing, Migration [9]
Squid Ink	#232F3E	Primary dark brand background [9]

Table 1: AWS Service Category Color Palette

For theme modes: light mode uses Background #FFFFFF with Foreground #000000 [9], while dark mode uses Background #000000 with Foreground #FFFFFF and arrow color #9BA7B6 [9]. Dark backgrounds are recommended for in-person presentations, light for web content [18, 19].

3.2 Typography

Amazon Ember is the master brand font with variants including Ember Display (headlines >72pt), Ember Condensed, Ember Mono, and Ember Duospace [10, 20]. The typesetting hierarchy specifies: Headlines in Amazon Ember Display Light (sentence case), Subheads in Amazon Ember Bold ($\leqslant$ half headline point size), and Body copy in Amazon Ember Light (left-aligned, max 15 words per line) [10]. No more than 10 consecutive words should be set in bold [10].

For architecture diagrams specifically, all label text must be **12pt Arial** throughout [11]. For monospace contexts (code slides), the fallback is Monaco, Menlo, Consolas, Courier Prime [21].

3.3 Layout Rules

All AWS presentations use **16:9 widescreen** format at 13.33 $\times$ 7.5 inches (1920 $\times$ 1080 pixels) [7, 8]. Architecture diagrams follow strict rules: 2pt line weights, Open Arrow Size 4 connectors, 0.05” nested group buffers, and accessibility-tested colors that must not be altered [11]. Icons must be used at predefined size, color, and format without cropping, flipping, rotating, or reshaping [11].

4 Skill Architecture and Module Structure

The PPTX generation skill follows the dominant **three-layer architecture** pattern separating template management, data binding via JSON, and a rendering engine [1]. This design ensures that visual changes don’t trigger code deployments, reducing error surface [1].

4.1 Three-Layer Architecture

The system comprises an **Input Layer** (JSON configuration + template .pptx + data sources), an **Engine Layer** (validation, orchestration, and rendering via python-pptx), and an **Output Layer** (generated .pptx with post-generation validation) [1, 3]. The FastAPI + PostgreSQL + python-pptx service pattern provides the proven microservice foundation, containerized with Docker [3].

4.2 Module Organization

The recommended codebase structure follows separation of concerns demonstrated by production PPTX generators [3, 22]:

```
aws_pptx_skill/
├── main.py                    # FastAPI application entry point
├── config/
│   ├── aws_theme.py           # Color constants, font specs, dimensions
│   └── slide_types.py         # Slide type enum and metadata
├── schemas/
│   ├── presentation.py        # Pydantic models for presentation config
│   └── slides.py              # Per-slide-type Pydantic schemas
├── templates/
│   ├── aws_dark.pptx          # Master template (dark background)
│   ├── aws_light.pptx         # Master template (light background)
│   └── manifest.json          # Template manifest with layout/placeholder map
├── engine/
│   ├── orchestrator.py        # Director pattern - coordinates generation
│   ├── style_engine.py        # Applies AWS fonts, colors, fills
│   └── validator.py           # Pre/post-generation validation
├── builders/
│   ├── base.py                # Abstract SlideBuilder interface
│   ├── cover_builder.py       # Title/Cover slide
│   ├── content_builder.py     # Content/Bullet slide
│   ├── section_builder.py     # Section Divider slide
│   ├── architecture_builder.py # Architecture Diagram slide
│   ├── table_builder.py       # Table slide
│   └── chart_builder.py       # Chart/Data visualization slide
├── utils/
│   ├── units.py               # EMU/Inches/Pt conversion helpers
│   └── xml_helpers.py         # Direct XML manipulation for borders
└── tests/
    └── test_generation.py     # Validation and compliance tests
```

4.3 Key Design Patterns

Strategy Pattern for Slide Builders: Each slide type (title, content, chart, table) implements a common `SlideBuilder` interface, allowing the orchestrator to dispatch to the appropriate builder based on configuration [5]. This enables adding new slide types without modifying existing code.

Manifest-Based Template Validation: The pptx-cli approach validates output before it reaches humans by extracting compatibility warnings, inspecting layouts/placeholders/themes, and estimating text placeholder capacity [12]. The four-phase pipeline is: (1) Initialize from template, (2) Extract manifest of layouts/placeholders/rules, (3) Generate within approved boundaries, (4) Validate output [12].

Pydantic Schema Enforcement: Using Pydantic BaseModel classes to define slide configuration schemas ensures type safety and generates JSON Schema documentation automatically [4]. The Slide Master → Layout → Placeholder inheritance model in python-pptx maps naturally to this validation hierarchy [23].

Template-as-Starting-Point: The recommended approach uses a real .pptx file (with slides removed) as the template, preserving theme, slide master, and slide layouts that control presentation appearance [6]. Placeholder idx values remain stable throughout the slide' s life and are consistent across all slides using the same layout [13].

5 AWS Slide Type Definitions and Layouts

AWS technical presentations follow a consistent structure across various session formats. Breakout sessions are lecture-style presentations covering architecture, best practices, and deep dives into AWS services [24]. Session content accommodates all levels of familiarity from beginner (level 100) to expert (level 400) [25]. The presentation flow typically includes introduction, agenda, context, architecture overview, deep-dive content, demonstrations, summary, resources, and Q&A sections.

Slide Type	Purpose	Key Visual Elements
Title/Cover	Opens presentation, identifies session	Session code, title, speaker information
Agenda	Provides session roadmap	Numbered list of 3–5 topics, ample whitespace
Section Divider	Marks topic transitions	Bold title centered on colored/dark background
Content/Bullet	Explains concepts and features	Heading + 3–5 bullets, left-aligned text
Architecture Diagram	Visualizes system design	Full-width diagram with AWS icons, arrows, grouping containers [16]
Two-Column Split	Combines diagram with explanation	Left: diagram/image; Right: bullet points
Demo/Code	Shows implementation details	Code block with monospace font on dark background
Customer Story	Highlights real-world usage	Customer logo, quote, key metrics
Comparison/Table	Contrasts options or features	Table or side-by-side columns
Summary/Takeaways	Recaps key points	3–5 numbered takeaways
Resources/CTA	Directs next steps	Links, QR codes, related session codes
Q&A/Closing	Ends presentation	"Questions?" with speaker contact info

Table 2: AWS Standard Slide Types

Content Level System: Session depth is indicated by level codes — Level 100 (broad overview), Level 200 (best practices with demos), Level 300 (comprehensive deep dives), and Level 400 (expert-level intricacies) [25]. Session codes use a track prefix followed by a three-digit number (e.g., BOA208 = Build on AWS, Level 200) [26].

Layout Principles: Architecture diagrams use full-bleed or near-full-bleed layouts with the diagram occupying most of the slide area. Text-heavy explanation is kept on separate slides to maintain visual clarity. Diagrams use standardized symbols connected by arrows indicating data flow, with services grouped into logical containers for VPCs, subnets, regions, and accounts [27]. AWS provides a standardized Speaker PowerPoint Template and AWS Fonts package to all re:Invent presenters [28].

6 Core Engine Implementation

This section provides the complete reference implementation for the core engine components. All code uses python-pptx 1.0.2 [2] with precise EMU-based positioning [29].

6.1 AWS Theme Constants Module

```
# aws_theme.py - AWS Brand Constants
from pptx.util import Inches, Pt, Emu, Cm
from pptx.dml.color import RGBColor
from pptx.enum.text import PP_ALIGN

# === AWS Colors ===
SQUID_INK = RGBColor(0x23, 0x2F, 0x3E)        # #232F3E - Dark background [[9](https://github.
SMILE_ORANGE = RGBColor(0xED, 0x71, 0x00)      # #ED7100 - Compute/accent [[9](https://github.
GALAXY_PURPLE = RGBColor(0x8C, 0x4F, 0xFF)     # #8C4FFF - Analytics [[9](https://github.com/a
NEBULA = RGBColor(0xC9, 0x25, 0xD1)            # #C925D1 - Database [[9](https://github.com/aw
MARS_RED = RGBColor(0xDD, 0x34, 0x4C)          # #DD344C - Security [[9](https://github.com/aw
COSMOS_PINK = RGBColor(0xE7, 0x15, 0x7B)       # #E7157B - App Integration [[9](https://github
ENDOR_GREEN = RGBColor(0x7A, 0xA1, 0x16)       # #7AA116 - Storage [[9](https://github.com/aws
ORBIT_TURQUOISE = RGBColor(0x01, 0xA8, 0x8D)   # #01A88D - AI/ML [[9](https://github.com/awsla
WHITE = RGBColor(0xFF, 0xFF, 0xFF)
LIGHT_GRAY = RGBColor(0x7D, 0x89, 0x98)        # #7D8998 - Borders [[9](https://github.com/aws

# === Typography ===
```

```
FONT_PRIMARY = "Amazon Ember"       # Master brand font [[10](https://developer.amazon.com/en
FONT_FALLBACK = "Arial"             # Fallback for diagrams [[11](s3://research-documents-pro
FONT_MONO = "Consolas"              # Code slides [[21](https://cloudscape.design/foundation/

TITLE_SIZE = Pt(36)
SUBTITLE_SIZE = Pt(20)
BODY_SIZE = Pt(16)
CAPTION_SIZE = Pt(12)
DIAGRAM_LABEL_SIZE = Pt(12)         # Required for architecture diagrams

# === Slide Dimensions (16:9 Widescreen) ===
SLIDE_WIDTH = Inches(13.333)        # 16:9 widescreen [[7](https://support.microsoft.com/en-u
SLIDE_HEIGHT = Inches(7.5)          # 16:9 widescreen [[7](https://support.microsoft.com/en-u

# === Layout Constants ===
MARGIN_LEFT = Inches(0.75)
MARGIN_TOP = Inches(1.2)
CONTENT_WIDTH = Inches(11.8)
CONTENT_HEIGHT = Inches(5.5)
```

6.2 Presentation Initialization

```
# engine/orchestrator.py
from pptx import Presentation
from pptx.util import Inches
from config.aws_theme import *

class PresentationOrchestrator:
    """Coordinates PPTX generation from structured configuration [[1](https://dzone.com/artic

    def __init__(self, template_path: str = None):
        # Load from template preserving masters/layouts [[17](https://python-pptx.readthedocs
        if template_path:
            self.prs = Presentation(template_path)
        else:
            self.prs = Presentation()

        # Set 16:9 widescreen dimensions [[14](https://scanny-python-pptx.mintlify.app/api/pr
        self.prs.slide_width = SLIDE_WIDTH
        self.prs.slide_height = SLIDE_HEIGHT

        # Register slide builders [[5](https://github.com/MiniMax-AI/skills/blob/main/skills/
        self._builders = {}

    def register_builder(self, slide_type: str, builder):
        self._builders[slide_type] = builder

    def generate(self, config: dict) -> str:
        """Generate presentation from validated config."""
        for slide_config in config["slides"]:
            slide_type = slide_config["type"]
            builder = self._builders[slide_type]
            builder.build(self.prs, slide_config)

        output_path = config.get("output_path", "output.pptx")
        self.prs.save(output_path)
        return output_path
```

6.3 Style Engine (Backgrounds and Fills)

Slide backgrounds support solid fills and gradient fills [31, 32]. The gradient angle controls direction: 270° = top-to-bottom [33], and stops range from 0.0 to 1.0 [32].

```
# engine/style_engine.py
from pptx.util import Pt, Inches
from pptx.dml.color import RGBColor
from config.aws_theme import *

class AWSStyleEngine:
    """Applies AWS brand styling to slides and shapes."""

    @staticmethod
    def apply_dark_background(slide):
        """Apply Squid Ink dark background [[9](https://github.com/awslabs/aws-icons-for-plant
        bg = slide.background
        fill = bg.fill
        fill.solid()
        fill.fore_color.rgb = SQUID_INK

    @staticmethod
    def apply_gradient_background(slide, color1=SQUID_INK,
                                  color2=RGBColor(0x16, 0x1E, 0x2D)):
        """Apply dark gradient background [[32](https://python-pptx.readthedocs.io/en/stable/
        bg = slide.background
        fill = bg.fill
        fill.gradient()
        fill.gradient_angle = 270.0  # Top to bottom [[33](https://scanny-python-pptx.mintlify
        stops = fill.gradient_stops
        stops.color.rgb = color1
        stops.position = 0.0
        stops[[19](s3://research-documents-prod-iad/arn:aws:ds:us-east-1:764946308314:user/d-9
        stops[[19](s3://research-documents-prod-iad/arn:aws:ds:us-east-1:764946308314:user/d-9

    @staticmethod
    def format_title(text_frame, text, size=TITLE_SIZE,
                     color=WHITE, bold=True):
        """Apply AWS title formatting [[34](https://python-pptx.readthedocs.io/en/latest/user/
        text_frame.clear()
        p = text_frame.paragraphs
        p.alignment = PP_ALIGN.LEFT
        run = p.add_run()
        run.text = text
        run.font.name = FONT_PRIMARY
        run.font.size = size
        run.font.bold = bold
        run.font.color.rgb = color

    @staticmethod
    def format_body_text(text_frame, bullets, size=BODY_SIZE,
                         color=WHITE):
        """Apply AWS body text with bullets [[34](https://python-pptx.readthedocs.io/en/latest
        text_frame.clear()
        for i, bullet in enumerate(bullets):
            p = text_frame.paragraphs if i == 0 else text_frame.add_paragraph()
            p.level = 0
            p.alignment = PP_ALIGN.LEFT
            run = p.add_run()
            run.text = bullet
            run.font.name = FONT_PRIMARY
            run.font.size = size
```

```
            run.font.color.rgb = color
```

6.4 Table Creation with AWS Styling

Table cell borders require XML manipulation as they are not exposed in the high-level API [35, 36]. Cell fills use the standard `cell.fill.solid()` API [37].

```
# utils/xml_helpers.py - Table border XML manipulation [[35](https://github.com/scanny/python
from pptx.oxml.ns import qn
from pptx.util import Pt
from lxml import etree

def set_cell_border(cell, side, color_hex="7D8998", width_pt=1):
    """Set cell border via XML manipulation [[35](https://github.com/scanny/python-pptx/issues
    tc = cell._tc
    tcPr = tc.get_or_add_tcPr()

    side_map = {"left": "a:lnL", "right": "a:lnR",
                "top": "a:lnT", "bottom": "a:lnB"}

    ln = etree.SubElement(tcPr, qn(side_map[side]))
    ln.set("w", str(int(Pt(width_pt))))
    ln.set("cap", "flat")
    ln.set("cmpd", "sng")

    solidFill = etree.SubElement(ln, qn("a:solidFill"))
    srgbClr = etree.SubElement(solidFill, qn("a:srgbClr"))
    srgbClr.set("val", color_hex)
```

7 Slide Builder Reference Scripts

This section provides complete, production-ready builder implementations for each AWS slide type. All builders inherit from a common `SlideBuilder` abstract base class and implement the Strategy Pattern [5].

7.1 Base Builder Interface

```
# builders/base.py
from abc import ABC, abstractmethod
from pptx import Presentation
from engine.style_engine import AWSStyleEngine

class SlideBuilder(ABC):
    """Abstract base for all slide type builders."""

    def __init__(self):
        self.style = AWSStyleEngine()

    @abstractmethod
    def build(self, prs: Presentation, config: dict) -> None:
        """Build a slide from configuration dict."""
        pass

    def _get_layout(self, prs, layout_idx):
        """Get slide layout by index [[38](https://python-pptx.readthedocs.io/en/latest/user/s
        return prs.slide_layouts[layout_idx]
```

7.2 Cover/Title Slide Builder

```
# builders/cover_builder.py
from pptx.util import Inches, Pt
from pptx.enum.text import PP_ALIGN
```

```python
from config.aws_theme import *
from builders.base import SlideBuilder

class CoverSlideBuilder(SlideBuilder):
    """AWS Title/Cover slide with dark gradient background [[26](https://community.aws/posts/c

    def build(self, prs, config):
        slide = prs.slides.add_slide(prs.slide_layouts[[38](https://python-pptx.readthedocs.io
        self.style.apply_gradient_background(slide)

        # Session code (top-left, small orange text)
        if "session_code" in config:
            txBox = slide.shapes.add_textbox(
                MARGIN_LEFT, Inches(0.5), Inches(3), Inches(0.5))
            self.style.format_title(
                txBox.text_frame, config["session_code"],
                size=Pt(14), color=SMILE_ORANGE, bold=True)

        # Main title (large, white, left-aligned)
        txBox = slide.shapes.add_textbox(
            MARGIN_LEFT, Inches(2.5), Inches(10), Inches(2))
        self.style.format_title(
            txBox.text_frame, config["title"],
            size=Pt(40), color=WHITE)

        # Subtitle / Speaker info
        txBox = slide.shapes.add_textbox(
            MARGIN_LEFT, Inches(4.8), Inches(8), Inches(1.5))
        tf = txBox.text_frame
        tf.clear()
        p = tf.paragraphs
        run = p.add_run()
        run.text = config.get("subtitle", "")
        run.font.name = FONT_PRIMARY
        run.font.size = Pt(18)
        run.font.color.rgb = LIGHT_GRAY

        # Orange accent bar (bottom)
        shape = slide.shapes.add_shape(
            1, Inches(0), Inches(7.2), SLIDE_WIDTH, Inches(0.08))  # 1=RECTANGLE
        shape.fill.solid()
        shape.fill.fore_color.rgb = SMILE_ORANGE
        shape.line.fill.background()
```

7.3 Section Divider Builder

```python
# builders/section_builder.py
from pptx.util import Inches, Pt
from pptx.enum.text import PP_ALIGN
from config.aws_theme import *
from builders.base import SlideBuilder

class SectionDividerBuilder(SlideBuilder):
    """Bold section title centered on dark background."""

    def build(self, prs, config):
        slide = prs.slides.add_slide(prs.slide_layouts[[38](https://python-pptx.readthedocs.io
        self.style.apply_dark_background(slide)

        # Centered section title
        txBox = slide.shapes.add_textbox(
```

```
            Inches(1.5), Inches(2.8), Inches(10.3), Inches(2))
        tf = txBox.text_frame
        tf.clear()
        p = tf.paragraphs
        p.alignment = PP_ALIGN.CENTER
        run = p.add_run()
        run.text = config["title"]
        run.font.name = FONT_PRIMARY
        run.font.size = Pt(44)
        run.font.bold = True
        run.font.color.rgb = WHITE

        # Optional subtitle
        if "subtitle" in config:
            p2 = tf.add_paragraph()
            p2.alignment = PP_ALIGN.CENTER
            run2 = p2.add_run()
            run2.text = config["subtitle"]
            run2.font.name = FONT_PRIMARY
            run2.font.size = Pt(20)
            run2.font.color.rgb = SMILE_ORANGE
```

7.4 Content/Bullet Slide Builder

```
# builders/content_builder.py
from pptx.util import Inches, Pt
from config.aws_theme import *
from builders.base import SlideBuilder

class ContentSlideBuilder(SlideBuilder):
    """Standard content slide with heading + bullets [[34](https://python-pptx.readthedocs.io/

    def build(self, prs, config):
        slide = prs.slides.add_slide(prs.slide_layouts[[38](https://python-pptx.readthedocs.i
        self.style.apply_dark_background(slide)

        # Slide heading
        txBox = slide.shapes.add_textbox(
            MARGIN_LEFT, Inches(0.6), CONTENT_WIDTH, Inches(0.8))
        self.style.format_title(
            txBox.text_frame, config["heading"],
            size=Pt(28), color=WHITE)

        # Bullet content area
        txBox = slide.shapes.add_textbox(
            MARGIN_LEFT, MARGIN_TOP + Inches(0.3),
            CONTENT_WIDTH, CONTENT_HEIGHT)
        tf = txBox.text_frame
        tf.word_wrap = True
        tf.clear()

        for i, bullet in enumerate(config.get("bullets", [])):
            p = tf.paragraphs if i == 0 else tf.add_paragraph()
            p.level = bullet.get("level", 0)
            p.space_after = Pt(8)
            run = p.add_run()
            run.text = bullet["text"] if isinstance(bullet, dict) else bullet
            run.font.name = FONT_PRIMARY
            run.font.size = BODY_SIZE
            run.font.color.rgb = WHITE
```

7.5 Architecture Diagram Slide Builder

```python
# builders/architecture_builder.py
from pptx.util import Inches, Pt
from config.aws_theme import *
from builders.base import SlideBuilder

class ArchitectureSlideBuilder(SlideBuilder):
    """Full-width architecture diagram slide [[16](https://aws.amazon.com/architecture/icons/

    def build(self, prs, config):
        slide = prs.slides.add_slide(prs.slide_layouts[[38](https://python-pptx.readthedocs.i
        self.style.apply_dark_background(slide)

        # Slide title (compact, top)
        txBox = slide.shapes.add_textbox(
            MARGIN_LEFT, Inches(0.3), CONTENT_WIDTH, Inches(0.6))
        self.style.format_title(
            txBox.text_frame, config["title"],
            size=Pt(24), color=WHITE)

        # Architecture diagram image (full-bleed area) [[39](https://python-pptx.readthedocs.
        img_path = config["diagram_path"]
        # Calculate centered position for diagram
        diagram_top = Inches(1.1)
        diagram_height = Inches(6.0)
        diagram_width = Inches(12.0)
        diagram_left = (SLIDE_WIDTH - diagram_width) / 2

        slide.shapes.add_picture(
            img_path, int(diagram_left), int(diagram_top),
            width=int(diagram_width), height=int(diagram_height))
```

7.6 Table Slide Builder

```python
# builders/table_builder.py
from pptx.util import Inches, Pt
from pptx.dml.color import RGBColor
from config.aws_theme import *
from utils.xml_helpers import set_cell_border
from builders.base import SlideBuilder

class TableSlideBuilder(SlideBuilder):
    """AWS-styled table with header row and alternating colors [[40](https://python-pptx.read

    def build(self, prs, config):
        slide = prs.slides.add_slide(prs.slide_layouts[[38](https://python-pptx.readthedocs.i
        self.style.apply_dark_background(slide)

        # Title
        txBox = slide.shapes.add_textbox(
            MARGIN_LEFT, Inches(0.4), CONTENT_WIDTH, Inches(0.6))
        self.style.format_title(
            txBox.text_frame, config["title"], size=Pt(24), color=WHITE)

        # Table data
        headers = config["headers"]
        rows_data = config["rows"]
        n_rows = len(rows_data) + 1  # +1 for header
        n_cols = len(headers)
```

```
    table_frame = slide.shapes.add_table(
        n_rows, n_cols,
        MARGIN_LEFT, Inches(1.3),
        CONTENT_WIDTH, Inches(5.5))
    table = table_frame.table

    # Style header row [[37](https://github.com/scanny/python-pptx/issues/52)]
    for col_idx, header in enumerate(headers):
        cell = table.cell(0, col_idx)
        cell.text = header
        cell.fill.solid()
        cell.fill.fore_color.rgb = SMILE_ORANGE
        # Format text
        for paragraph in cell.text_frame.paragraphs:
            for run in paragraph.runs:
                run.font.name = FONT_FALLBACK
                run.font.size = Pt(11)
                run.font.bold = True
                run.font.color.rgb = WHITE
        # Borders [[35](https://github.com/scanny/python-pptx/issues/71)]
        for side in ["top", "bottom", "left", "right"]:
            set_cell_border(cell, side, "FFFFFF", 0.5)

    # Style data rows (alternating) [[37](https://github.com/scanny/python-pptx/issues/52
    for row_idx, row_data in enumerate(rows_data):
        for col_idx, value in enumerate(row_data):
            cell = table.cell(row_idx + 1, col_idx)
            cell.text = str(value)
            cell.fill.solid()
            if row_idx % 2 == 0:
                cell.fill.fore_color.rgb = RGBColor(0x2A, 0x3A, 0x4E)
            else:
                cell.fill.fore_color.rgb = RGBColor(0x1E, 0x2C, 0x3E)
            for paragraph in cell.text_frame.paragraphs:
                for run in paragraph.runs:
                    run.font.name = FONT_FALLBACK
                    run.font.size = Pt(10)
                    run.font.color.rgb = WHITE
```

8 JSON Configuration Schema and Data Interface

The skill uses a **configuration-driven approach** where JSON declarations map to python-pptx operations [1]. Pydantic BaseModel classes define and validate all input schemas, ensuring type safety and auto-generating JSON Schema documentation [4].

8.1 Pydantic Schema Definitions

```python
# schemas/presentation.py
from pydantic import BaseModel, Field
from typing import List, Optional, Literal
from enum import Enum

class SlideType(str, Enum):
    COVER = "cover"
    SECTION_DIVIDER = "section_divider"
    CONTENT = "content"
    ARCHITECTURE = "architecture"
    TABLE = "table"
    CHART = "chart"
    TWO_COLUMN = "two_column"
    CODE = "code"
```

```
    SUMMARY = "summary"
    QA = "qa"

class BulletItem(BaseModel):
    text: str
    level: int = Field(default=0, ge=0, le=3)

class CoverSlideConfig(BaseModel):
    type: Literal["cover"]
    title: str = Field(..., max_length=100)
    subtitle: Optional[str] = None
    session_code: Optional[str] = None
    speaker_name: Optional[str] = None

class ContentSlideConfig(BaseModel):
    type: Literal["content"]
    heading: str = Field(..., max_length=80)
    bullets: List[BulletItem] = Field(..., min_length=1, max_length=8)

class ArchitectureSlideConfig(BaseModel):
    type: Literal["architecture"]
    title: str
    diagram_path: str  # Path to diagram image
    caption: Optional[str] = None

class TableSlideConfig(BaseModel):
    type: Literal["table"]
    title: str
    headers: List[str] = Field(..., min_length=2)
    rows: List[List[str]] = Field(..., min_length=1)

class ChartSlideConfig(BaseModel):
    type: Literal["chart"]
    title: str
    chart_type: Literal["bar", "line", "pie", "column"]
    categories: List[str]
    series: List[dict]  # [{name: str, values: List[float]}]

class PresentationConfig(BaseModel):
    """Root configuration for entire presentation [[4](https://pydantic.dev/articles/llm-intro
    title: str
    template: str = Field(default="aws_dark",
                          description="Template name from registry")
    theme: Literal["dark", "light"] = "dark"
    slides: List[dict] = Field(..., min_length=1)
    output_path: str = "output.pptx"
    metadata: Optional[dict] = None
```

8.2 JSON Input Example

```
{
  "title": "AWS Migration Strategy",
  "template": "aws_dark",
  "theme": "dark",
  "output_path": "migration_strategy.pptx",
  "slides": [
    {
      "type": "cover",
      "title": "Migrating to AWS: A 6R Strategy",
      "subtitle": "John Smith, Solutions Architect",
      "session_code": "ARC301"
```

```
    },
    {
      "type": "content",
      "heading": "The 6 R's of Migration",
      "bullets": [
        {"text": "Rehost — Lift and shift to EC2", "level": 0},
        {"text": "Replatform — Lift, tinker, and shift", "level": 0},
        {"text": "Repurchase — Move to SaaS", "level": 0},
        {"text": "Refactor — Re-architect for cloud-native", "level": 0},
        {"text": "Retire — Decommission unused apps", "level": 0},
        {"text": "Retain — Keep on-premises for now", "level": 0}
      ]
    },
    {
      "type": "architecture",
      "title": "Target Architecture",
      "diagram_path": "diagrams/target_arch.png"
    },
    {
      "type": "table",
      "title": "Migration Timeline",
      "headers": ["Phase", "Duration", "Workloads", "Status"],
      "rows": [
        ["Discovery", "4 weeks", "All", "Complete"],
        ["Wave 1", "6 weeks", "Web tier", "In Progress"],
        ["Wave 2", "8 weeks", "Data tier", "Planned"]
      ]
    }
  ]
}
```

8.3 Template Registry and Manifest

The template registry maps template names to .pptx files and their manifests [12, 6]. The manifest documents available layouts, placeholder idx values (which remain stable throughout the slide's life [13]), and validation rules:

```python
# engine/template_registry.py
import json
from pathlib import Path
from pptx import Presentation

class TemplateRegistry:
    """Repository pattern for template management [[6](https://python-pptx.readthedocs.io/en/

    def __init__(self, templates_dir: str = "templates/"):
        self.templates_dir = Path(templates_dir)
        self._registry = {}
        self._load_manifests()

    def _load_manifests(self):
        for manifest_path in self.templates_dir.glob("*/manifest.json"):
            with open(manifest_path) as f:
                manifest = json.load(f)
            self._registry[manifest["name"]] = {
                "path": manifest_path.parent / manifest["template_file"],
                "layouts": manifest["layouts"],
                "placeholders": manifest["placeholders"],
            }

    def get_template(self, name: str) -> Presentation:
```

```
        """Load template preserving masters/layouts [[17](https://python-pptx.readthedocs.io/
        entry = self._registry[name]
        return Presentation(str(entry["path"]))

    def validate_config(self, name: str, config: dict) -> list:
        """Validate config against template manifest [[12](https://pypi.org/project/pptx-cli/
        errors = []
        entry = self._registry[name]
        for slide in config.get("slides", []):
            slide_type = slide.get("type")
            if slide_type not in entry["layouts"]:
                errors.append(f"Unknown slide type: {slide_type}")
        return errors
```

8.4 REST API Contract

The service exposes a multi-step workflow via FastAPI [3]:

```
# main.py
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from schemas.presentation import PresentationConfig

app = FastAPI(title="AWS PPTX Generation Skill")

@app.post("/presentations/generate")
async def generate_presentation(config: PresentationConfig):
    """Generate AWS-compliant PPTX from configuration."""
    orchestrator = PresentationOrchestrator(
        template_path=registry.get_template_path(config.template))
    # Register all builders
    for slide_type, builder_cls in BUILDER_MAP.items():
        orchestrator.register_builder(slide_type, builder_cls())

    output = orchestrator.generate(config.dict())
    return FileResponse(output, media_type="application/vnd.openxmlformats-officedocument.pres

@app.get("/templates")
async def list_templates():
    """List available templates with their capabilities."""
    return registry.list_all()
```

The API supports file-like object I/O via BytesIO streams for network-based workflows without filesystem interaction [6], enabling integration with S3 storage or streaming responses.

9 Deployment and Quality Assurance

9.1 Validation Pipeline

The skill implements a **four-phase validation pipeline** to ensure output quality [12]: (1) Initialize from a real template with pre-configured masters, (2) Extract a manifest of layouts, placeholders, assets, and rules, (3) Generate only within approved boundaries, (4) Validate output before delivery. Pre-generation validation via manifest inspects layouts, placeholders, themes, and estimates text placeholder capacity [12].

```
# engine/validator.py
from pptx import Presentation
from pptx.util import Inches, Pt
from config.aws_theme import *

class PresentationValidator:
    """Post-generation compliance checker [[12](https://pypi.org/project/pptx-cli/)]."""

    CHECKS = [
```

```
        "slide_dimensions",    # Must be 16:9
        "background_color",    # Must use approved palette
        "font_compliance",     # Must use Ember/Arial
        "color_compliance",    # Only approved AWS colors
        "text_overflow",       # No text exceeding bounds
    ]

    def validate(self, pptx_path: str) -> dict:
        prs = Presentation(pptx_path)
        results = {"passed": [], "failed": [], "warnings": []}

        # Check dimensions
        if (prs.slide_width == SLIDE_WIDTH and
            prs.slide_height == SLIDE_HEIGHT):
            results["passed"].append("slide_dimensions")
        else:
            results["failed"].append("slide_dimensions: not 16:9")

        # Check each slide
        for idx, slide in enumerate(prs.slides):
            self._check_slide(slide, idx, results)

        return results

    def _check_slide(self, slide, idx, results):
        """Validate individual slide compliance."""
        # Check fonts on all text frames
        for shape in slide.shapes:
            if shape.has_text_frame:
                for para in shape.text_frame.paragraphs:
                    for run in para.runs:
                        if run.font.name and run.font.name not in [
                            FONT_PRIMARY, FONT_FALLBACK, FONT_MONO,
                            "Amazon Ember Display", "Amazon Ember Bold"
                        ]:
                            results["warnings"].append(
                                f"Slide {idx}: non-AWS font '{run.font.name}'")
```

9.2 Error Handling Strategy

The skill separates design and logic to isolate errors — template changes don’t trigger code deployments [1]. Key error handling patterns include Pydantic validation at the API boundary (catching malformed input before generation begins [4]), placeholder idx stability for reliable access across all slides using the same layout [13], and file-like object I/O via BytesIO for network workflows without filesystem dependencies [6].

9.3 Deployment Architecture

The recommended deployment uses Docker containerization with the FastAPI service pattern [3]:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

For AWS-native deployment, the service integrates with **Amazon ECS/Fargate** for serverless container orchestration, **S3** for template and output storage, **API Gateway** for REST endpoint management, and **CloudWatch** for monitoring and logging. The BytesIO stream support [6] enables direct S3 upload/download without local filesystem interaction.

9.4 Template Management Best Practices

Templates should be created in PowerPoint with all desired masters, layouts, and placeholder positions configured visually, then loaded as the starting point [17, 6]. The manifest.json file documents each layout's available placeholders with their idx values, types, and dimensions. When updating templates, validate that placeholder idx values remain consistent [13] and run the full validation pipeline against sample configurations before deployment.

10 References

1. https://dzone.com/articles/scalable-document-generation-api
2. http://pypi.python.org/pypi/python-pptx
3. https://github.com/ysskrishna/ai-ppt-slide-generator
4. https://pydantic.dev/articles/llm-intro
5. https://github.com/MiniMax-AI/skills/blob/main/skills/pptx-generator/SKILL.md
6. https://python-pptx.readthedocs.io/en/latest/user/presentations.html
7. https:// support.microsoft.com/ en-us/ office/ change-the-size-of-your-slides-040a811c-be43-40b9-8d04-0de5ed79987e
8. https://www.brandwares.com/Design4PowerPoint.php
9. https://github.com/awslabs/aws-icons-for-plantuml/blob/main/AWSSymbols.md
10. https://developer.amazon.com/en-US/alexa/branding/echo-guidelines/identity-guidelines/typography
11. AWS-Architecture-Icons-Deck_For-Light-BG_07312025.pdf
12. https://pypi.org/project/pptx-cli/
13. https://python-pptx.readthedocs.io/en/latest/user/placeholders-using.html
14. https://scanny-python-pptx.mintlify.app/api/presentation
15. https://developer.amazon.com/en-US/alexa/branding/alexa-guidelines/brand-guidelines/color
16. https://aws.amazon.com/architecture/icons/
17. https://python-pptx.readthedocs.io/en/latest/user/concepts.html
18. https://cloudscape.design/foundation/visual-foundation/visual-modes/
19. AWS-Architecture-Icons-Deck_For-Dark-BG_07312025.pdf
20. http://luc.devroye.org/fonts-89157.html
21. https://cloudscape.design/foundation/visual-foundation/typography/
22. https://pypi.org/project/python-pptx-interface/
23. https://python-pptx.readthedocs.io/en/latest/dev/analysis/placeholders/index.html
24. https://aws.amazon.com/blogs/compute/the-serverless-attendees-guide-to-aws-reinvent-2024/
25. https://aws.amazon.com/events/summits/los-angeles/faqs/
26. https://community.aws/posts/decoding-aws-reinvent-a-guide-for-first-timers
27. https://www.processon.io/blog/what-is-aws-architecture-diagram
28. https://aws-sponsorship.zendesk.com/hc/en-us/articles/27118914997652-re-Invent-2024-Speaker-PowerPoint-Template-and-AWS-Fonts
29. https://python-pptx.readthedocs.io/en/latest/user/autoshapes.html
30. https://python-pptx.readthedocs.io/
31. https://python-pptx.readthedocs.io/en/latest/dev/analysis/sld-background.html
32. https://python-pptx.readthedocs.io/en/stable/dev/analysis/dml-gradient.html
33. https://scanny-python-pptx.mintlify.app/api/fill
34. https://python-pptx.readthedocs.io/en/latest/user/text.html

35. https://github.com/scanny/python-pptx/issues/71

36. https:// stackoverflow.com/ questions/ 42610829/ python-pptx-changing-table-style-or-adding-borders-to-cells/48295481

37. https://github.com/scanny/python-pptx/issues/52

38. https://python-pptx.readthedocs.io/en/latest/user/slides.html

39. https://python-pptx.readthedocs.io/en/latest/user/quickstart.html

40. https://python-pptx.readthedocs.io/en/latest/user/table.html